

School of Computer Science and Engineering

Winter Semester 2025-26

Embedded Systems Course Project

Active-Safety Helmet for Rider Safety & Monitoring

Parizaa Munshi - 23BCE0760
Maan Hitesh Teckwani - 23BCE0810
Jothi Baskar - 23BCT0036

Objectives

- To transform conventional protective headgear into an intelligent safety system capable of monitoring both environmental conditions and the rider’s physiological state.
- A fail-safe ignition lock mechanism is proposed to ensure that the vehicle operates only when the helmet is properly worn.
- An automated dynamic fit system is designed to adjust internal tension in real time based on aerodynamic forces and rider movement.

Detailed Functional Architecture

1. Sensing Layer

- **Rider Detection & Helmet Usage:** IR Proximity Sensor detects if a rider's head is inside the helmet. Sends signal to controller to enable/disable bike ignition via relay.
- **Motion & Fatigue Monitoring:** IMU (accelerometer + gyroscope + magnetometer) tracks head tilt, nodding, and sudden jerks to detect drowsiness or unusual movements.
- **Collision & Obstacle Detection:** Ultrasonic sensors positioned at blind spots and rear measure distance to nearby objects to prevent collisions.
- **Helmet Fit Measurement:** Force Sensitive Resistors (FSRs) or Velostat pressure sheet detect strap tightness and pressure on the head for adaptive fit actuation.

2. Processing Layer

- **Controller:** ESP32 handles all sensor interfacing, real-time data processing, and optional Bluetooth communication.
- **Sensor Data Acquisition:** Continuously reads data from IR, IMU, ultrasonic sensors, and FSR/pressure sheet with high-frequency sampling.
- **Signal Conditioning:** Filters sensor noise and converts raw readings into meaningful metrics such as head position, acceleration, tilt, obstacle distance, and strap pressure.
- **Risk & Event Detection:**
  - Helmet off → IR sensor triggers relay to lock bike ignition.
  - Drowsiness/fatigue → IMU detects repeated nodding/tilting → triggers vibration alert.
  - Accident → IMU detects sudden acceleration/jerk → triggers emergency response.
  - Collision → Ultrasonic sensor detects object within threshold → triggers LED visual warning.
- **Adaptive Fit Control:** Uses FSR readings to determine helmet strap tightness and sends commands to micro servo motors to adjust fit automatically.

3. Actuation & Alert Layer

- **Ignition Control:** Relay module disables or enables bike ignition based on helmet detection.
- **Haptic Feedback:** Vibration motor alerts rider in case of drowsiness or fatigue.
- **Visual Alerts:** LEDs illuminate to warn of obstacles or collision threats.
- **Adaptive Fit Actuation:** Micro servo motors adjust helmet strap tension based on FSR/pressure sheet data, ensuring optimal comfort and safety.
- **Emergency Response:** If an accident is detected, the system can send location/alert to predefined contacts via BLE or optional mobile app.

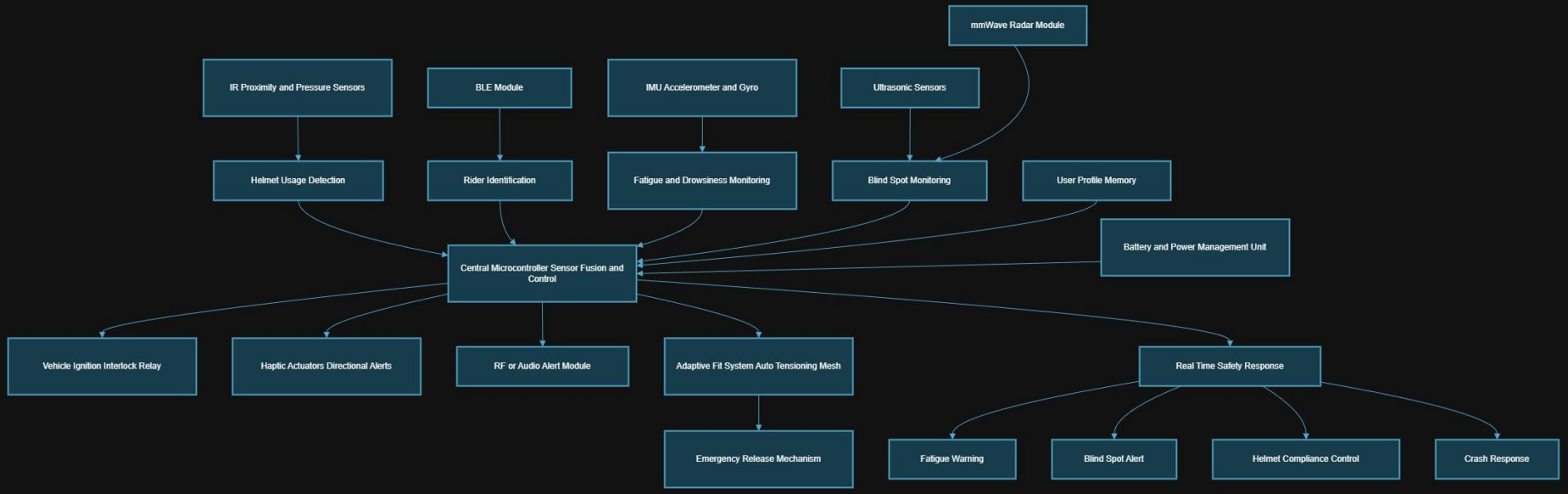
4. Communication Layer

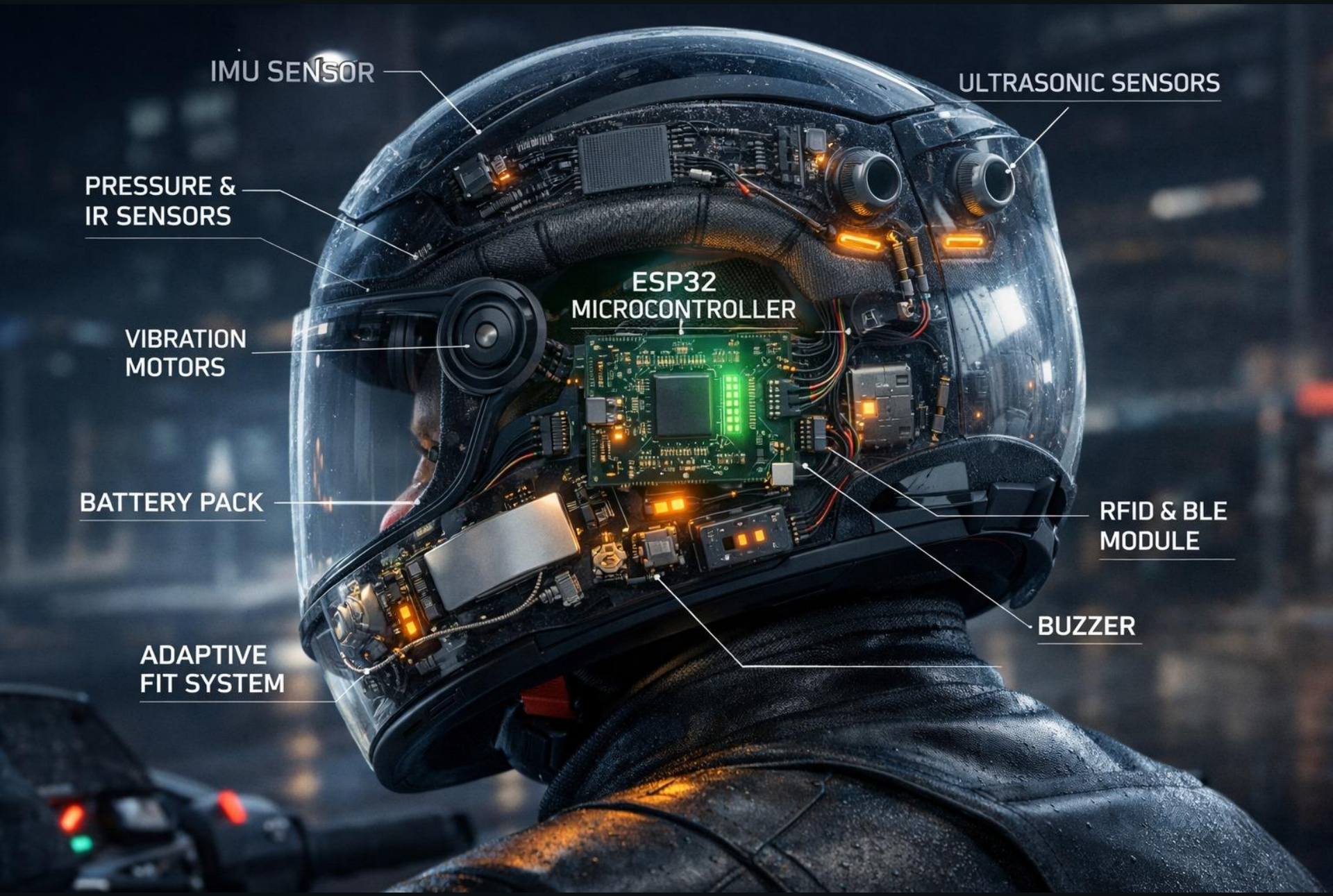
- **BLE Connectivity:** ESP32 (built-in or external module) transmits real-time sensor data, alerts, and helmet status to a mobile device.
- **Mobile App Interface:** Receives notifications for drowsiness, collision warnings, or accidents, and can log helmet usage history.

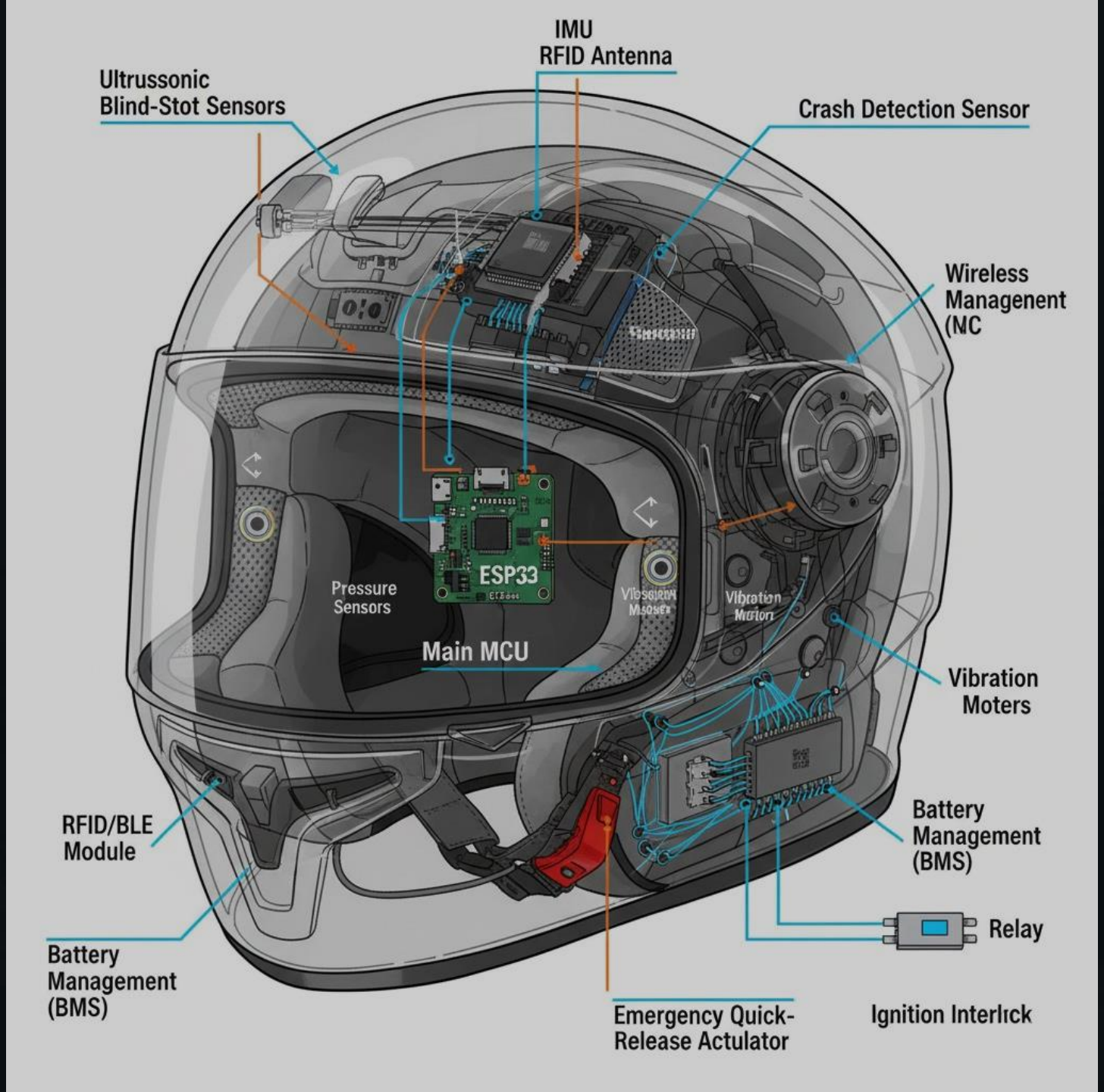
5. Overall System Flow

- Sensors continuously monitor rider, helmet fit, motion, and surroundings.
- ESP32 processes the data in real-time to detect risks or unsafe conditions.
- Depending on the detected event, actuators respond: vibration, LEDs, ignition relay, or servo motors for strap adjustment.
- Optional BLE communication relays data and alerts to a mobile app for monitoring or emergency response.

Detailed Block Diagram




IMU SENSOR
ULTRASONIC SENSORS
PRESSURE &
IR SENSORS
ESP32
MICROCONTROLLER
VIBRATION
MOTORS
BATTERY PACK
RFID & BLE
MODULE
BUZZER
ADAPTIVE
FIT SYSTEM


IMU
RFID Antenna
Ultrussonic
Blind-Stot Sensors
Crash Detection Sensor
Wireless
Managenent
(MC
ESP33
Pressure
Sensors
Main MCU
Vibration
Moters
RFID/BLE
Module
Battery
Management
(BMS)
Relay
Battery
Management
(BMS)
Emergency Quick-
Release Actulator
Ignition Interlick

Hardware Selection & Justification

1. ESP32 (Main Controller)

Justification:

- Built-in **Wi-Fi and BLE**, so no separate Bluetooth module is needed.
- Multiple **ADC and digital I/O pins** to interface with IR sensor, IMU, FSR, ultrasonic sensors, LEDs, and relays.
- Powerful enough to perform **real-time sensor data processing** for risk detection and adaptive actuation.
- Low power consumption suitable for wearable applications.
- Wide community support and libraries for sensors and actuators.

2. Proximity Sensor

Justification:

- Accurately detects the presence of a head inside the helmet.
- Compact and low-power, suitable for mounting inside helmets.
- Provides an analog output compatible with ESP32 ADC pins for distance detection.

3. MPU-6050 IMU (Inertial Measurement Unit)

Justification:

- Tracks head movements (tilt, nodding) and sudden impacts for **drowsiness and accident detection**.
- Communicates via **I2C**, making it easy to interface with ESP32.
- Small size, low power, widely available, and well-supported libraries for sensor fusion.
- Magnetometer in MPU-9250 adds optional orientation awareness if needed.

4. Ultrasonic Sensors

Justification:

- Measures distance to nearby obstacles for **collision prevention**.
- Easy to interface with ESP32 using digital I/O pins.
- Affordable, reliable, and widely used for proximity detection in automotive and robotics applications.
- Can be mounted at blind spots and rear of the helmet or bike.

5. Relay Module

Justification:

- Electrically isolates the ESP32 from high-power bike ignition circuitry.
- Enables **automatic ignition lock** if helmet is not worn.
- Simple to control using a GPIO pin from ESP32.

6. LEDs

Justification:

- Provides **visual alerts** for collision warnings or status indicators.
- Low power, easy to control via ESP32 GPIO.
- Optional RGB LEDs allow flexible indication patterns.

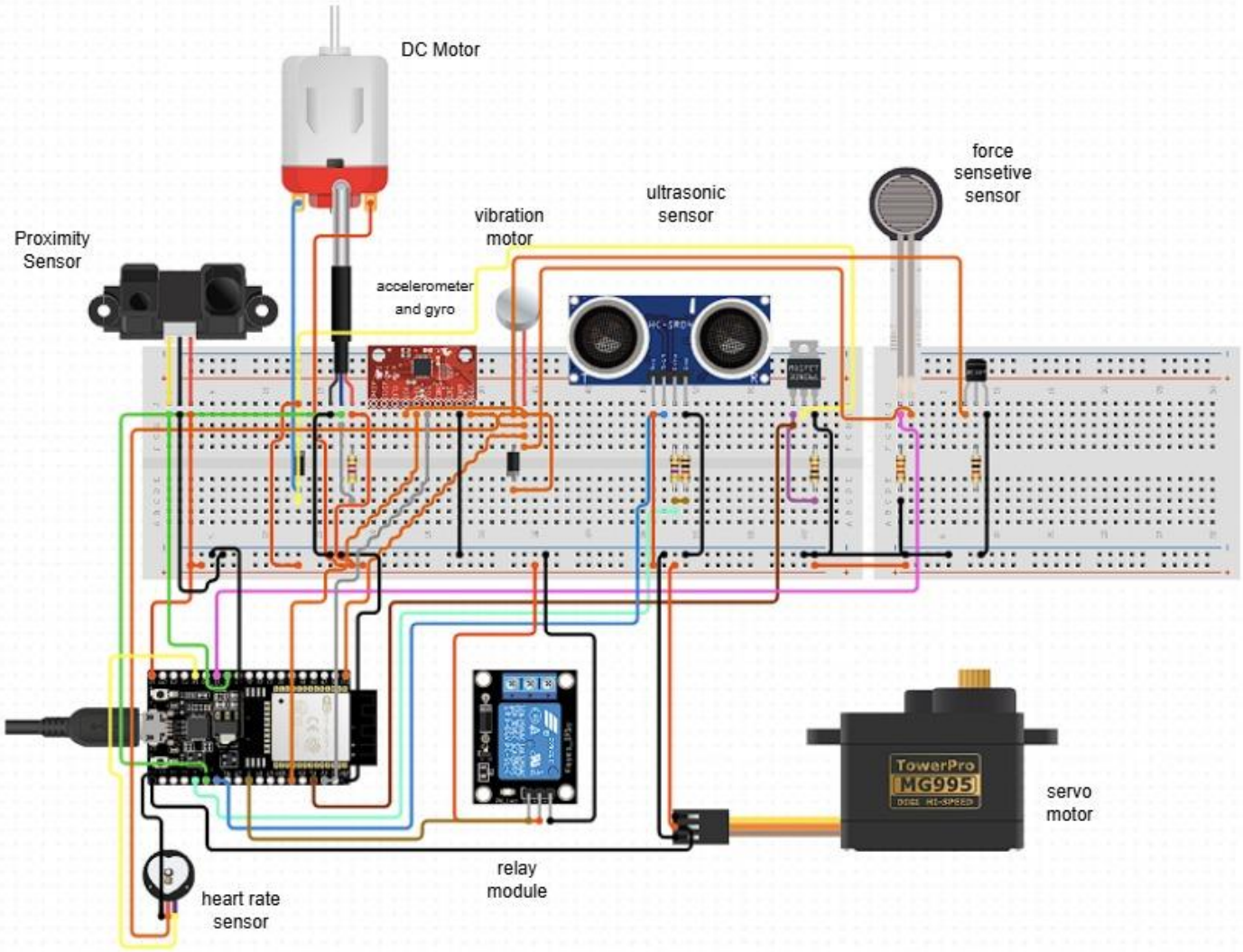
7. Haptic Vibration Motor

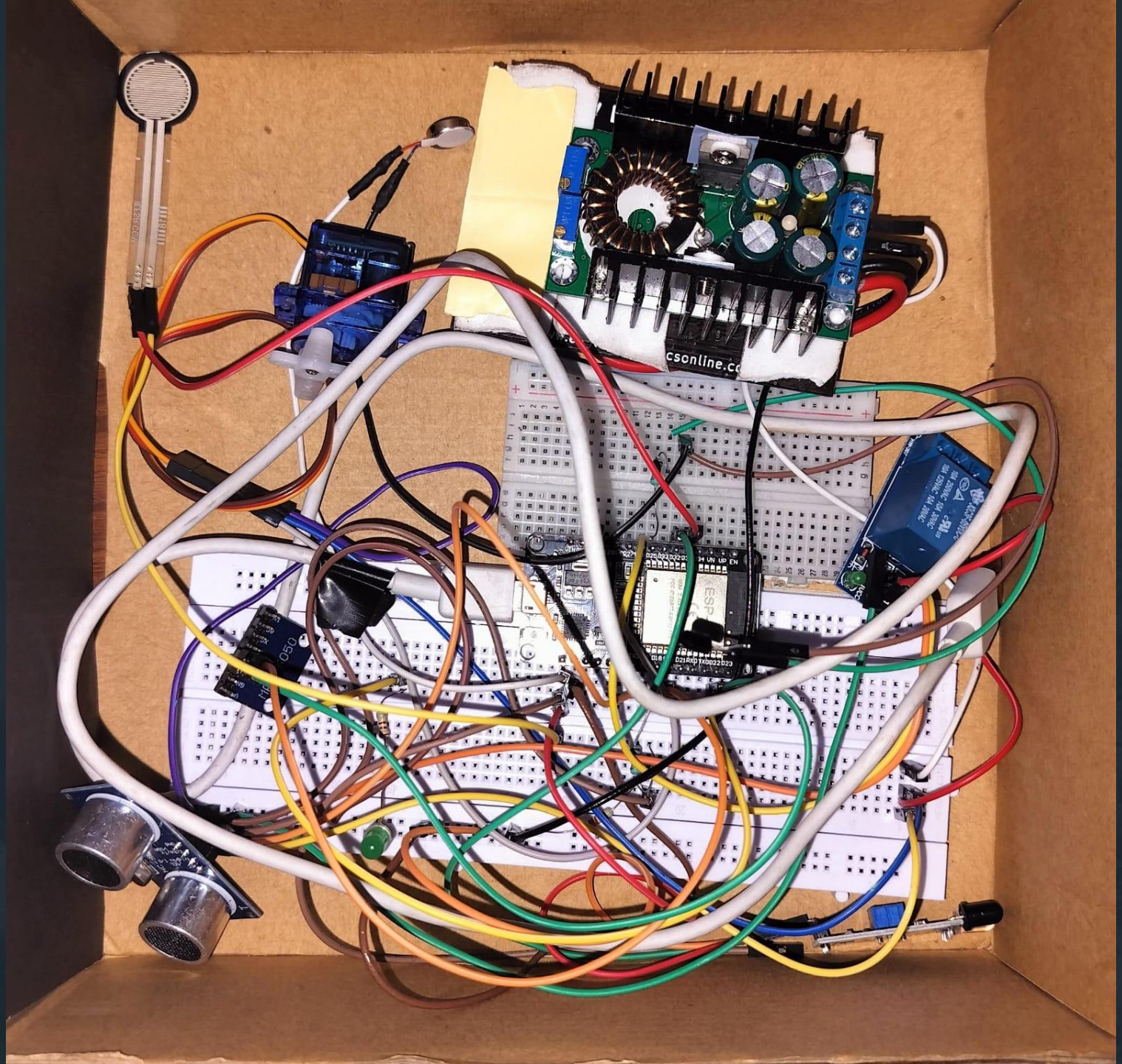
Justification:

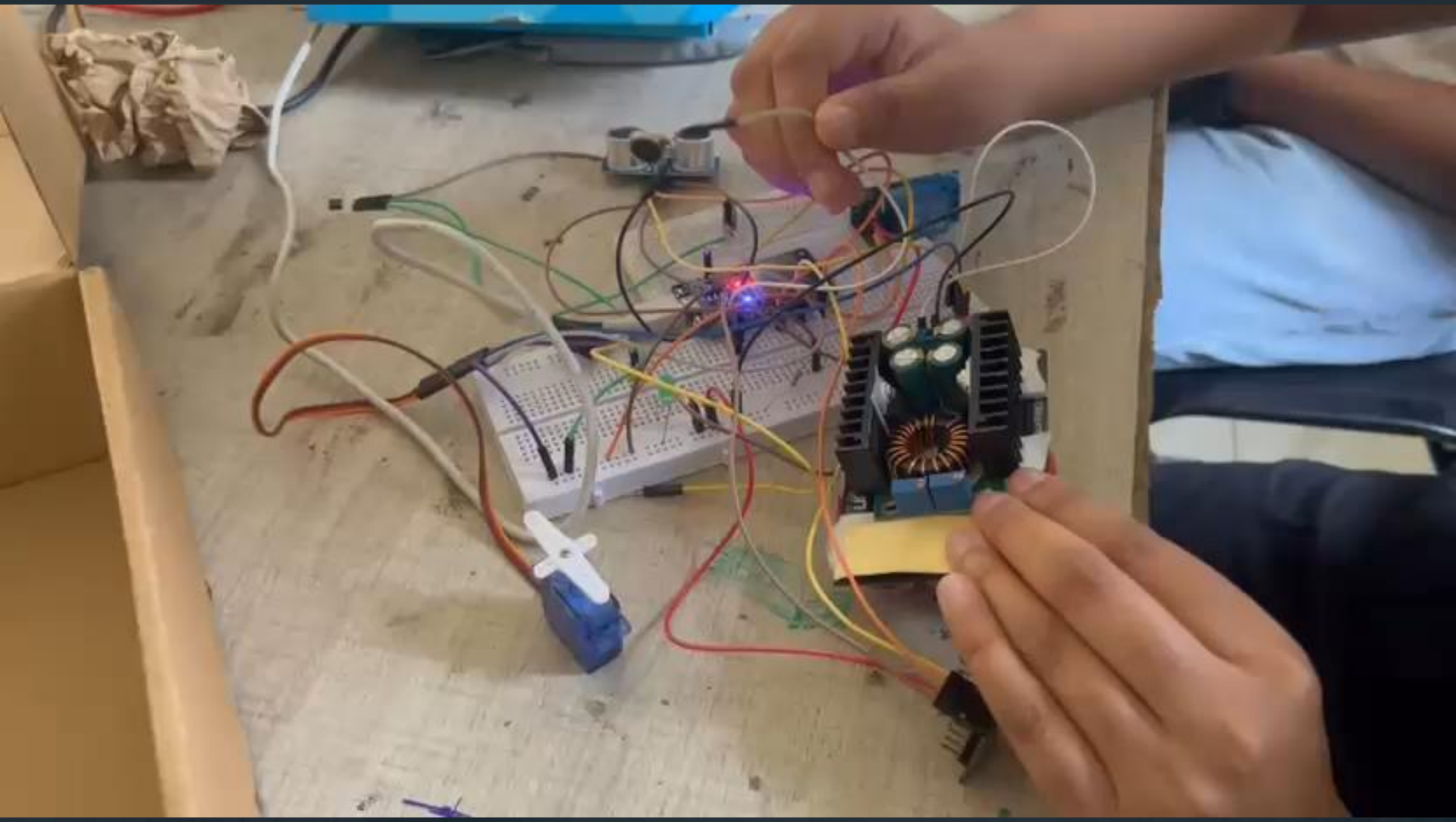
- Provides **drowsiness/fatigue alerts** through vibration feedback.
- Compact and low power, can be embedded in helmet padding.
- Can be controlled directly via a transistor or motor driver from ESP32.

8. Auto-Tension Adaptive Mesh System

- **Flexible Cable / Mesh Layer:** Nylon thread (lightweight, strong, flexible)
- **Force Sensitive Resistor (FSR):** Interlink FSR 402 or Velostat pressure sheet as alternative
- **Micro Servo Motors:** TowerPro SG90 or MG90S for strap adjustment
- **Servo Driver (if >2 servos):** PCA9685 16-channel PWM driver
  Justification:
- FSR provides **feedback on strap tightness**, enabling adaptive fit.
- Nylon thread provides durable, flexible support for adjustable tension.
- Small micro servos are lightweight and capable of adjusting straps automatically.
- Servo driver allows **control of multiple servos** with a single ESP32 without overloading PWM pins.


DC Motor
force
sensetive
sensor
ultrasonic
sensor
vibration
motor
Proximity
Sensor
accelerometer
and gyro
TowerPro
MG995
servo
motor
relay
module
heart rate
sensor





Software Selection

•IDE: Arduino IDE / PlatformIO – Easy development and debugging for ESP32. PlatformIO adds advanced project management.

•Firmware: Embedded C/C++ – Low-level hardware control, fast execution, real-time task handling.

•Mobile App: Flutter / React Native – Cross-platform apps to receive alerts, monitor helmet status, and view sensor data.

•Cloud (optional): Firebase – Stores logs, supports push notifications, enables analytics and remote monitoring.

•RTOS: FreeRTOS (ESP32 built-in) – Schedules multiple tasks deterministically for safety-critical operations like helmet detection, drowsiness alerts, collision warnings, and servo actuation.

Feasibility Analysis

- **Rider Detection & Helmet Usage:** High feasibility. IR sensor detects head presence; ESP32 handles Bluetooth and relay-controlled ignition lock. Main challenge: false readings in bright light.
- **Fatigue / Drowsiness Monitoring:** Medium feasibility. IMU detects head nodding; vibration alerts the rider. Challenge: differentiating normal movements from fatigue.
- **Accident Detection:** Medium feasibility. IMU can sense sudden impacts; thresholds need careful tuning to avoid false alarms.
- **Collision Prevention:** Medium feasibility. Ultrasonic sensors detect obstacles and trigger LED alerts. Challenge: performance in rain, dust, or traffic clutter.
- **Auto-Tension Adaptive Mesh:** Feasible. FSR/Velostat measures tightness; micro servos adjust tension. Challenge: mechanical design and durability.
- **Components:** All (IR sensor, relay, ESP32, IMU, LEDs, ultrasonic sensors, FSR, servos) are widely available and compatible with ESP32.

Risk Management

- **Hardware Risks:** Sensor errors, servo failure, relay malfunction. Mitigation: calibration, durable parts, fail-safe overrides.
- **Software Risks:** False positives from IMU, ESP32 memory limits, Bluetooth dropouts. Mitigation: optimized code, reconnection logic, threshold tuning.
- **Safety Risks:** Alerts distracting rider, system failure during ride. Mitigation: safe defaults, gentle vibration.
- **Operational Risks:** Limited battery life, mechanical wear. Mitigation: rechargeable battery, durable materials.
- **Regulatory Risks:** Helmet modifications affecting safety certification or legal compliance. Mitigation: prototype testing, clear disclaimers.

Initial Coding / Simulation

1) Blind Spot Detection + Alert

Ultrasonic Distance Measurement:

TRIG_PIN sends a short ultrasonic pulse.

ECHO_PIN receives the pulse after it bounces off an object.

readDistance() calculates the distance in centimeters using the pulse travel time.

Haptic Alert:

VIB_PIN controls a vibration motor.

If an object is detected within 10–80 cm, the motor turns on to alert the rider.

Serial Output:

Distance readings are printed to the Serial Monitor for debugging.

Loop Timing:

The sensor reads distance every 150 milliseconds.

The code continuously measures the distance to obstacles behind or beside the rider and vibrates a motor if something is dangerously close.

```
//Blind-Spot Detection Using Ultrasonic + Haptic Alert
#define TRIG_PIN 5
#define ECHO_PIN 18
#define VIB_PIN 25

long duration;

float readDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  duration = pulseIn(ECHO_PIN, HIGH, 30000);
  return duration * 0.034 / 2.0;
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(VIB_PIN, OUTPUT);
}

void loop() {
  float d = readDistance();

  digitalWrite(VIB_PIN, d < 80 && d > 10);

  Serial.println(d);
  delay(150);
}
```

2) Helmet Detection and Relay

Helmet Sensor:

HELMET_SENSOR reads an analog signal (e.g., from an IR proximity sensor) to detect if the helmet is worn.

A threshold of 1500 determines if the helmet is on.

Relay Control:

RELAY_PIN controls a relay connected to the bike's ignition.

If the helmet is detected (helmetOn == true), the relay is activated (ignition allowed).

If not, the relay stays off (ignition disabled).

Serial Output:

Prints true or false to indicate helmet status.

Loop Timing:

The sensor is checked every 200 milliseconds.

The ESP32 continuously checks if the helmet is worn and only allows the bike ignition if it is.

```
#define HELMET_SENSOR 34
#define RELAY_PIN 14

void setup() {
  Serial.begin(115200);
  pinMode(RELAY_PIN, OUTPUT);
}

void loop() {
  bool helmetOn = analogRead(HELMET_SENSOR) > 1500;
  digitalWrite(RELAY_PIN, helmetOn ? HIGH : LOW);

  Serial.println(helmetOn);
  delay(200);
}
```

3) Crash/Impact Detection

IMU Initialization:

MPU6050 imu reads accelerometer data along X, Y, and Z axes.

Acceleration Measurement:

imu.getAcceleration(&ax, &ay, &az) retrieves the current acceleration values.

The **magnitude of acceleration** is calculated as $\text{sqrt}(ax^2 + ay^2 + az^2)$.

Crash Detection:

If the magnitude exceeds 35000, the code considers it a **crash or strong impact**.

Alerts:

BUZZER and VIB (vibration motor) are turned on if a crash is detected.

Serial Output:

Prints the acceleration magnitude for monitoring/debugging.

Loop Timing:

Measurements are taken every 80 milliseconds for near real-time detection.

The ESP32 continuously monitors acceleration and triggers a buzzer and vibration motor if a sudden high impact (crash) occurs.

```
3  //Crash / Impact Detection Using IMU
4  #include <Wire.h>
5  #include <MPU6050.h>
6
7  MPU6050 imu;
8
9  #define BUZZER 27
10 #define VIB 25
11
12 void setup() {
13   Serial.begin(115200);
14   Wire.begin();
15   imu.initialize();
16   pinMode(BUZZER, OUTPUT);
17   pinMode(VIB, OUTPUT);
18 }
19
20 void loop() {
21   int16_t ax, ay, az;
22   imu.getAcceleration(&ax, &ay, &az);
23
24   float mag = sqrt(ax*ax + ay*ay + az*az);
25
26   bool crash = mag > 35000;
27
28   digitalWrite(BUZZER, crash);
29   digitalWrite(VIB, crash);
30
31   Serial.println(mag);
32   delay(80);
33 }
34
35
36
```

4) Drowsiness Detection

IMU Initialization:

MPU6050 imu reads acceleration along X, Y, and Z axes.

Movement Detection:

The acceleration magnitude is calculated as $\text{sqrt}(ax^2 + ay^2 + az^2)$.

If the magnitude exceeds 20000, it considers the rider **active** and updates lastMove with the current time.

Drowsiness Alert:

If **no significant movement is detected for more than 6 seconds**, the buzzer is turned on, alerting the rider.

Serial Output:

Prints the acceleration magnitude for monitoring/debugging.

Loop Timing:

Checks for movement every 120 milliseconds.

The ESP32 monitors head movement and sounds a buzzer if the rider's head stays almost motionless for 6+ seconds, indicating potential drowsiness.

```
2  //Fatigue / Drowsiness Detection Using IMU
3  #include <Wire.h>
4  #include <MPU6050.h>
5
6  MPU6050 imu;
7
8  #define BUZZER 27
9
10 unsigned long lastMove = 0;
11
12 void setup() {
13   Serial.begin(115200);
14   Wire.begin();
15   imu.initialize();
16   pinMode(BUZZER, OUTPUT);
17 }
18
19 void loop() {
20   int16_t ax, ay, az;
21   imu.getAcceleration(&ax, &ay, &az);
22
23   float mag = sqrt(ax*ax + ay*ay + az*az);
24
25   if (mag > 20000) lastMove = millis();
26
27   digitalWrite(BUZZER, millis() - lastMove > 6000);
28
29   Serial.println(mag);
30   delay(120);
31 }
32
```

5) Blind Spot Detection and Haptic Alerts

- **Ultrasonic Sensors:**
  - TRIG_L/ECHO_L measure distance on the left side.
  - TRIG_R/ECHO_R measure distance on the right side.
  - getDist() sends a pulse and calculates distance in centimeters.
- **Haptic Alerts:**
  - VIB_L vibrates if an object is detected 10–70 cm on the left.
  - VIB_R vibrates if an object is detected 10–70 cm on the right.
- **Serial Output:**
  - Prints left and right distances for debugging.
- **Loop Timing:**
  - Checks distances every 150 milliseconds for near real-time alerts.
- **In short:** The ESP32 continuously monitors both sides of the rider and vibrates the corresponding motor if a vehicle or obstacle is detected in the blind spot.

```
//Dual-Side Blind-Spot Detection (Left / Right)
#define TRIG_L 5
#define ECHO_L 18
#define TRIG_R 19
#define ECHO_R 21

#define VIB_L 25
#define VIB_R 26

float getDist(int t, int e) {
  digitalWrite(t, LOW);
  delayMicroseconds(2);
  digitalWrite(t, HIGH);
  delayMicroseconds(10);
  digitalWrite(t, LOW);
  long d = pulseIn(e, HIGH, 30000);
  return d * 0.034 / 2.0;
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_L, OUTPUT);
  pinMode(ECHO_L, INPUT);
  pinMode(TRIG_R, OUTPUT);
  pinMode(ECHO_R, INPUT);
  pinMode(VIB_L, OUTPUT);
  pinMode(VIB_R, OUTPUT);
}

void loop() {
  float dl = getDist(TRIG_L, ECHO_L);
  float dr = getDist(TRIG_R, ECHO_R);

  digitalWrite(VIB_L, dl < 70 && dl > 10);
  digitalWrite(VIB_R, dr < 70 && dr > 10);

  Serial.print(dl);
  Serial.print(",");
  Serial.println(dr);

  delay(150);
}
```